K.N. Toosi University of Technology

OFFLINE JAVA GUIDE

Chapter 1
Arrays

A complete guide to Java arrays — declaration, initialization, iteration, built-in utility methods, sorting, searching, and multidimensional arrays.

COURSE
Advanced Programming

SEMESTER
Spring 2026

DEPARTMENT
Computer Engineering

v1.0

Table of Contents

1.1 What is an Array?

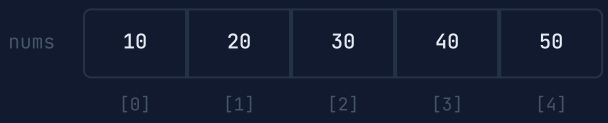
An **array** is a fixed-size, ordered collection of elements of the **same type**. It stores values in contiguous memory locations, giving you O(1) access to any element by its index.

Key Properties

PROPERTY	DETAIL
Fixed size	Once created, the length cannot change. You must decide the size at creation time.
Same type	All elements must be of the declared type (or a subtype).
Zero-indexed	The first element is at index `0`, the last at index `length - 1`.
Reference type	The array variable holds a **reference** to the array object on the heap, not the values themselves.
Default values	Numeric types default to `0`, `boolean` to `false`, objects to `null`.

How Arrays Live in Memory

MEMORY MODEL — INT[] NUMS = {10, 20, 30, 40, 50};



NOTE

Arrays are **objects** in Java. Even `int[]` is an object — it has a `.length` property and lives on the heap. This is different from C/C++ where arrays are raw memory blocks.

Array vs. ArrayList (Quick Look-Ahead)

FEATURE	ARRAY	ARRAYLIST
Size	Fixed	Dynamic (grows automatically)
Primitives	Yes (`int[]`)	No (only objects: `Integer`)
Performance	Faster	Slightly slower (overhead)
Syntax	`nums[0]`	`list.get(0)`

1.2 Declaring & Initializing

Declaration Syntax

Both forms are valid — brackets after type or after variable name:

```java
// Preferred style (brackets with type)
int[] numbers;
double[] grades;
String[] names;

// Also valid but less common
int numbers[];
String names[];
```

Method 1 — Array Literal (Best for Known Values)

```java
int[] nums = {10, 20, 30, 40, 50};
String[] days = {"Sat", "Sun", "Mon", "Tue", "Wed"};
double[] vals = {1.0, 2.5, 3.7};
```

Method 2 — new with Explicit Size

```java
int[] nums = new int[5];
// Creates: [0, 0, 0, 0, 0]  (default int values)

String[] names = new String[3];
// Creates: [null, null, null]  (default object values)

boolean[] flags = new boolean[4];
// Creates: [false, false, false, false]
```

Method 3 — new with Values

```java
int[] nums = new int[]{10, 20, 30};
// Useful when you want to pass an anonymous array to a method:
someMethod(new int[]{1, 2, 3});
```

MISTAKE

You **cannot** do both — specifying size and providing values:

`int[] nums = new int[3]{1, 2, 3}; // COMPILE ERROR`

Either let Java count: `new int[]{1, 2, 3}`, or specify the size: `new int[3]`.

Anonymous Arrays

Arrays without a variable name — useful for passing inline values to methods:

```java
// Passing values directly without creating a named variable
System.out.println(java.util.Arrays.toString(new int[]{5, 10, 15}));
```

Default Values Table

ARRAY TYPE	DEFAULT VALUE	AFTER `NEW T[N]`
`int[]`	`0`	`[0, 0, 0, ...]`
`double[]`	`0.0`	`[0.0, 0.0, ...]`
`boolean[]`	`false`	`[false, false, ...]`
`char[]`	`'\u0000'`	`['\0', '\0', ...]`
`String[]`	`null`	`[null, null, ...]`
Any object[]	`null`	`[null, null, ...]`

QUICK REFERENCE — DECLARATION CHEATSHEET

- Known values → `int[] a = {1, 2, 3};`
- Unknown values, known size → `int[] a = new int[100];`
- Pass inline to method → `method(new int[]{1, 2});`
- Size from variable → `int[] a = new int[n];` (n must be known at runtime)

1.3 Accessing & Modifying Elements

Reading Elements

```java
int[] nums = {10, 20, 30, 40, 50};

int first = nums[0];    // 10
int third = nums[2];    // 30
int last  = nums[4];    // 50 — or better: nums[nums.length - 1]
```

Modifying Elements

```java
nums[0] = 99;   // array is now: [99, 20, 30, 40, 50]
nums[2] = nums[0];  // array is now: [99, 20, 99, 40, 50]
```

The length Property

```java
int[] nums = {10, 20, 30};
System.out.println(nums.length);  // 3

// Use it for safe last-element access:
System.out.println(nums[nums.length - 1]);  // 30
```

MISTAKE

`length` is a **property** (no parentheses), not a method. Unlike `String.length()` which is a method with `()`:

`nums.length` ✓ `nums.length()` ✗ (compile error)

ArrayIndexOutOfBoundsException

Accessing an index outside `0` to `length - 1` throws this runtime exception:

```java
int[] nums = {10, 20, 30};
System.out.println(nums[3]);  // RUNTIME ERROR!
// Valid indices: 0, 1, 2  — 3 is out of bounds
```

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3
```

OUTPUT

WARNING

This is a **runtime** error, not a compile error. The compiler cannot catch it because the index might come from user input or a computation. Always validate indices before accessing, especially when they come from external data.

Safely Accessing the Last Element

JAVA

```java
int[] nums = {10, 20, 30};

// Dangerous — crashes if array is empty
int last = nums[nums.length - 1];

// Safe pattern — check first
if (nums.length > 0) {
    int last = nums[nums.length - 1];
}
```

1.4 Iterating Over Arrays

Classic for Loop (When You Need the Index)

```java
int[] nums = {10, 20, 30, 40, 50};

for (int i = 0; i < nums.length; i++) {
    System.out.println("Index " + i + ": " + nums[i]);
}
```

JAVA

```
Index 0: 10 Index 1: 20 Index 2: 30 Index 3: 40 Index 4: 50
```

OUTPUT

TIP Always use `i < nums.length` (not `<=`). The last valid index is `nums.length - 1`.

Enhanced for-each Loop (When You Don't Need the Index)

```java
int[] nums = {10, 20, 30, 40, 50};

for (int num : nums) {
    System.out.print(num + " ");
}
```

JAVA

```
10 20 30 40 50
```

OUTPUT

TRAP The for-each loop gives you a **read-only copy** of each element for primitives. You **cannot modify** the array through it:

`for (int num : nums) { num = 0; } // does NOT change nums!`

To modify elements, use the classic `for` loop with index access.

While Loop (Rare, but Valid)

```java
int[] nums = {10, 20, 30, 40, 50};
int i = 0;
while (i < nums.length) {
    System.out.print(nums[i] + " ");
    i++;
}
```

JAVA

When to Use Which Loop

LOOP	USE WHEN
`for (int i = 0; ...)`	You need the index (modifying, searching, comparing with neighbors)
`for (int x : arr)`	You only need to read each element (printing, summing, finding max)
`while`	Iteration count is unknown / depends on dynamic condition

Common Patterns

JAVA

```java
int[] nums = {3, 7, 2, 9, 5};

// Sum
int sum = 0;
for (int n : nums) sum += n;
// sum = 26

// Find maximum
int max = nums[0];
for (int n : nums) if (n > max) max = n;
// max = 9

// Find minimum
int min = nums[0];
for (int n : nums) if (n < min) min = n;
// min = 2

// Count occurrences
int count = 0;
for (int n : nums) if (n % 2 == 0) count++;
// count = 1 (only 2 is even)
```

TIP

When finding max/min, initialize with `nums[0]` (not `0` or `Integer.MIN_VALUE`). This handles negative numbers correctly and works for any array type.

1.5 The Arrays Utility Class

Java provides `java.util.Arrays` — a class full of static methods for common array operations. You need to import it: `import java.util.Arrays;`

Arrays.toString() — Print an Array

```java
int[] nums = {10, 20, 30, 40, 50};
System.out.println(Arrays.toString(nums));
```

```
[10, 20, 30, 40, 50]
```

> NOTE
> Printing an array directly with `System.out.println(nums)` shows something like `[I@1b6d3586` — the type and memory address. Always use `Arrays.toString()` for readable output.

Arrays.fill() — Set All Elements to a Value

```java
int[] nums = new int[5];
Arrays.fill(nums, 42);
System.out.println(Arrays.toString(nums));  // [42, 42, 42, 42, 42]

// Fill a range: from index 1 (inclusive) to 4 (exclusive)
Arrays.fill(nums, 1, 4, 99);
System.out.println(Arrays.toString(nums));  // [42, 99, 99, 99, 42]
```

Arrays.copyOf() & Arrays.copyOfRange()

```java
int[] original = {1, 2, 3, 4, 5};

// Copy entire array (can change length — pads with defaults or truncates)
int[] copy1 = Arrays.copyOf(original, 5);
System.out.println(Arrays.toString(copy1));  // [1, 2, 3, 4, 5]

int[] copy2 = Arrays.copyOf(original, 3);
System.out.println(Arrays.toString(copy2));  // [1, 2, 3]

int[] copy3 = Arrays.copyOf(original, 7);
System.out.println(Arrays.toString(copy3));  // [1, 2, 3, 4, 5, 0, 0]

// Copy a range: from index 1 (inclusive) to 4 (exclusive)
int[] copy4 = Arrays.copyOfRange(original, 1, 4);
System.out.println(Arrays.toString(copy4));  // [2, 3, 4]
```

System.arraycopy() — Fast In-Place Copy

JAVA

```java
int[] src = {1, 2, 3, 4, 5};
int[] dest = new int[5];

// System.arraycopy(src, srcPos, dest, destPos, length)
System.arraycopy(src, 1, dest, 0, 3);
System.out.println(Arrays.toString(dest));  // [2, 3, 4, 0, 0]
```

TIP

`System.arraycopy()` is faster than `Arrays.copyOf()` because it doesn't need to create a new array — it copies into an existing one. Use it when performance matters in tight loops.

Arrays.equals() — Compare Two Arrays

JAVA

```java
int[] a = {1, 2, 3};
int[] b = {1, 2, 3};
int[] c = {1, 2, 4};

System.out.println(a == b);                 // false (different objects)
System.out.println(Arrays.equals(a, b));    // true  (same content)
System.out.println(Arrays.equals(a, c));    // false
```

MISTAKE

Never use `==` to compare arrays. It checks if they are the **same object** in memory, not if they have the same contents. Always use `Arrays.equals()`.

Complete Methods Reference

METHOD	DESCRIPTION	RETURNS
`Arrays.toString(arr)`	String representation of the array	`String`
`Arrays.fill(arr, val)`	Fill entire array with `val`	`void`
`Arrays.fill(arr, from, to, val)`	Fill range `[from, to)`	`void`
`Arrays.copyOf(arr, newLen)`	Copy with new length (pads/truncates)	new array
`Arrays.copyOfRange(arr, from, to)`	Copy range `[from, to)`	new array
`Arrays.equals(a, b)`	Compare contents element by element	`boolean`
`Arrays.sort(arr)`	Sort entire array in-place	`void`
`Arrays.sort(arr, from, to)`	Sort range `[from, to)`	`void`
`Arrays.binarySearch(arr, key)`	Binary search (array must be sorted)	`int` index
`Arrays.mismatch(a, b)`	Index of first differing element	`int`

1.6 Sorting Arrays

Java provides built-in sorting through `Arrays.sort()`. It uses **Dual-Pivot Quicksort** for primitives and **TimSort** for objects — both are highly optimized.

Sort Entire Array (Ascending)

```java
import java.util.Arrays;

int[] nums = {40, 10, 50, 20, 30};
Arrays.sort(nums);
System.out.println(Arrays.toString(nums));
```

JAVA

```
[10, 20, 30, 40, 50]
```

OUTPUT

Sort a Range

```java
int[] nums = {40, 10, 50, 20, 30};
// Sort only indices 1 to 3 (exclusive)
Arrays.sort(nums, 1, 4);
System.out.println(Arrays.toString(nums));
```

JAVA

```
[40, 10, 20, 50, 30]
```

OUTPUT

Index `0` (value `40`) and index `4` (value `30`) are untouched.

Works with All Primitive Types

```java
double[] doubles = {3.3, 1.1, 2.2};
Arrays.sort(doubles);
// [1.1, 2.2, 3.3]

char[] chars = {'z', 'a', 'm', 'b'};
Arrays.sort(chars);
// [a, b, m, z]

String[] words = {"delta", "alpha", "charlie"};
Arrays.sort(words);
// [alpha, charlie, delta]  (lexicographic order)
```

JAVA

Sort in Descending Order

`Arrays.sort()` only sorts ascending. For descending, you have three options:

Option 1 — Sort then Reverse (Best for Primitives)

```java
int[] nums = {40, 10, 50, 20, 30};
Arrays.sort(nums);  // [10, 20, 30, 40, 50]

// Manual reverse
for (int i = 0; i < nums.length / 2; i++) {
    int temp = nums[i];
    nums[i] = nums[nums.length - 1 - i];
    nums[nums.length - 1 - i] = temp;
}
System.out.println(Arrays.toString(nums));
```

JAVA

```
[50, 40, 30, 20, 10]
```

OUTPUT

Option 2 — Wrapper Classes with Comparator (Objects Only)

```java
import java.util.Arrays;
import java.util.Collections;

Integer[] nums = {40, 10, 50, 20, 30};
Arrays.sort(nums, Collections.reverseOrder());
System.out.println(Arrays.toString(nums));
```

JAVA

```
[50, 40, 30, 20, 10]
```

OUTPUT

NOTE

This only works with **object arrays** (`Integer[]` , not `int[]`). Primitives cannot be used with `Collections.reverseOrder()` . The tradeoff is a small performance overhead from boxing/unboxing.

Option 3 — Custom Logic with a Loop

```java
int[] nums = {40, 10, 50, 20, 30};
int[] desc = new int[nums.length];
Arrays.sort(nums); // sort ascending first

// Copy in reverse order
for (int i = 0; i < nums.length; i++) {
    desc[i] = nums[nums.length - 1 - i];
}
System.out.println(Arrays.toString(desc));  // [50, 40, 30, 20, 10]
```

JAVA

Arrays.parallelSort() — For Large Arrays

```java
int[] nums = new int[1_000_000];
// fill with random values...

Arrays.parallelSort(nums);  // uses multiple CPU cores
```

JAVA

METHOD	ALGORITHM	WHEN TO USE
`Arrays.sort(arr)`	Dual-Pivot Quicksort	Small to medium arrays (most cases)
`Arrays.parallelSort(arr)`	Parallel sort (Fork-Join)	Large arrays (> 8192 elements)
`Arrays.sort(arr, comp)`	TimSort	Object arrays with custom ordering

TIP

For most homework and course exercises, `Arrays.sort()` is all you need. Use `parallelSort()` only when dealing with very large datasets where you can measure the speed difference.

QUICK REFERENCE — SORTING CHEATSHEET

- Ascending: `Arrays.sort(arr);` — works for all types
- Range sort: `Arrays.sort(arr, from, to);` — sorts `[from, to)`
- Descending (primitive): sort ascending, then reverse in a loop
- Descending (object): `Arrays.sort(arr, Collections.reverseOrder());`
- Large arrays: `Arrays.parallelSort(arr);`
- All sort methods modify the array **in-place** (no new array created)

1.7 Searching in Arrays

Linear Search — Works on Any Array

Check each element one by one. O(n) time. No prerequisites.

```java
int[] nums = {40, 10, 50, 20, 30};
int target = 50;
int index = -1;

for (int i = 0; i < nums.length; i++) {
    if (nums[i] == target) {
        index = i;
        break;
    }
}

if (index != -1) {
    System.out.println("Found at index " + index);  // Found at index 2
} else {
    System.out.println("Not found");
}
```

Arrays.binarySearch() — Fast but Requires Sorted Array

Uses binary search: O(log n) time. The array **must be sorted first**, otherwise the result is undefined.

```java
int[] nums = {10, 20, 30, 40, 50};

// Search for 30
int idx = Arrays.binarySearch(nums, 30);
System.out.println(idx);  // 2

// Search for 35 (not in array)
int idx2 = Arrays.binarySearch(nums, 35);
System.out.println(idx2);  // -4  (negative = not found)
```

Understanding the Return Value

RESULT	MEANING
`≥ 0`	Index of the found element
`< 0`	Not found. The actual insertion point is `-(result + 1)`

```java
int[] nums = {10, 20, 30, 40, 50};
int result = Arrays.binarySearch(nums, 35);

if (result ⩾ 0) {
    System.out.println("Found at: " + result);
} else {
    int insertPoint = -(result + 1);
    System.out.println("Not found. Would be inserted at index " + insertPoint);
    // Output: Not found. Would be inserted at index 3
}
```

Binary Search on a Range

```java
int[] nums = {50, 10, 30, 20, 40};
Arrays.sort(nums);  // [10, 20, 30, 40, 50]

// Search only in range [1, 4) → elements {20, 30, 40}
int idx = Arrays.binarySearch(nums, 1, 4, 30);
System.out.println(idx);  // 2
```

WARNING

If you call `Arrays.binarySearch()` on an **unsorted** array, you will get a wrong answer — **not** an error message. Always sort first, or use linear search if the array isn't sorted.

QUICK REFERENCE — WHEN TO USE WHICH SEARCH

- Array is unsorted or you're searching once → **Linear search** (simple, no setup)
- Array is already sorted or you'll search many times → **Binary search** (sort once, then O(log n) each search)
- You need the insertion point for a missing element → parse `-(result + 1)`

1.8 Multidimensional Arrays

A multidimensional array is an **array of arrays**. The most common is the 2D array (a matrix). Java supports any number of dimensions.

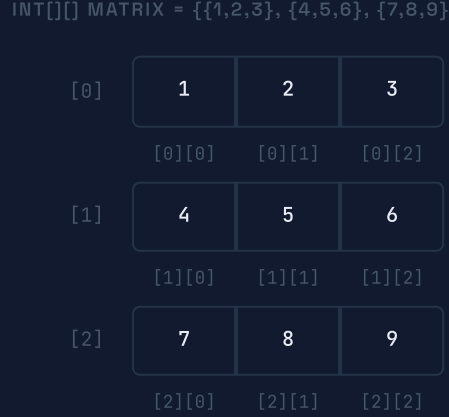
2D Array — Declaration & Initialization

```java
// Method 1: Literal
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

// Method 2: new with size
int[][] grid = new int[3][4];  // 3 rows, 4 columns — all zeros

// Method 3: new with values
int[][] data = new int[][]{
    {10, 20},
    {30, 40, 50}
};
```

Accessing Elements

```java
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

int val = matrix[1][2];   // 6  (row 1, column 2)
matrix[0][0] = 99;        // changes top-left to 99
```

Visual Layout



Iterating Over a 2D Array

```java
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

// Classic nested for loop
for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix[i].length; j++) {
        System.out.print(matrix[i][j] + " ");
    }
    System.out.println();
}

// Enhanced for-each (cleaner but no index)
for (int[] row : matrix) {
    for (int val : row) {
        System.out.print(val + " ");
    }
    System.out.println();
}
```

JAVA

```
1 2 3 4 5 6 7 8 9
```

OUTPUT

Printing with Arrays.deepToString()

```java
int[][] matrix = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
System.out.println(Arrays.deepToString(matrix));
```

JAVA

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

OUTPUT

MISTAKE

`Arrays.toString()` on a 2D array prints addresses of each row: `[[I@...]`. Use `Arrays.deepToString()` for multidimensional arrays.

Ragged Arrays (Jagged Arrays)

Each row can have a **different length**:

JAVA

```java
int[][] ragged = {
    {1, 2},
    {3, 4, 5, 6},
    {7, 8, 9}
};

// Always use matrix[i].length — NOT matrix[0].length
for (int i = 0; i < ragged.length; i++) {
    System.out.println("Row " + i + " has " + ragged[i].length + " elements");
}
```

OUTPUT

```
Row 0 has 2 elements Row 1 has 4 elements Row 2 has 3 elements
```

WARNING

When iterating a ragged array, always use `matrix[i].length` for the inner loop — **not** `matrix[0].length`. The latter assumes all rows have the same length and will crash on ragged arrays.

3D Arrays (Brief)

JAVA

```java
// 2 blocks, each 3x2
int[][][] cube = new int[2][3][2];

cube[0][1][0] = 42;
cube[1][2][1] = 99;

// Iteration
for (int i = 0; i < cube.length; i++) {
    for (int j = 0; j < cube[i].length; j++) {
        for (int k = 0; k < cube[i][j].length; k++) {
            System.out.print(cube[i][j][k] + " ");
        }
        System.out.println();
    }
    System.out.println("---");
}
```

Comparing Multidimensional Arrays

JAVA

```java
int[][] a = {{1, 2}, {3, 4}};
int[][] b = {{1, 2}, {3, 4}};

System.out.println(Arrays.equals(a, b));          // false (compares row references)
System.out.println(Arrays.deepEquals(a, b));      // true  (compares element by element)
```

1.9 Common Mistakes & Tips

1. Array Size Cannot Change

```java
int[] nums = {1, 2, 3};
nums[3] = 4;  // ArrayIndexOutOfBoundsException!

// To "add" an element, you must create a new array:
int[] bigger = Arrays.copyOf(nums, nums.length + 1);
bigger[3] = 4;  // [1, 2, 3, 4]
```

2. Negative Size

```java
int n = -5;
int[] arr = new int[n];  // NegativeArraySizeException
```

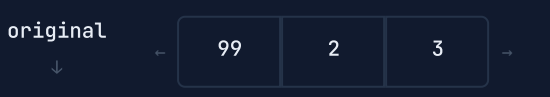
3. Confusing length with length()

```java
int[] arr = {1, 2, 3};
String s = "hello";

arr.length;    // correct — no parentheses
s.length();    // correct — with parentheses

arr.length();  // COMPILE ERROR
s.length;      // COMPILE ERROR
```

4. Assigning an Array Doesn't Copy It

```java
int[] original = {1, 2, 3};
int[] copy = original;  // NOT a copy — both point to the SAME array

copy[0] = 99;
System.out.println(original[0]);  // 99 — original was also changed!
```

WHY ALIASING HAPPENS — BOTH VARIABLES POINT TO THE SAME OBJECT



```java
// To actually copy:
int[] realCopy = Arrays.copyOf(original, original.length);
// or
int[] realCopy2 = original.clone();
```

5. Off-by-One in Loops

```java
int[] nums = {10, 20, 30};

// WRONG — ArrayIndexOutOfBoundsException
for (int i = 0; i ⩽ nums.length; i++) { }

// CORRECT
for (int i = 0; i < nums.length; i++) { }
```

6. Forgetting to Sort Before binarySearch

```java
int[] nums = {50, 10, 30, 20, 40};

// WRONG — undefined behavior (array is not sorted)
int idx = Arrays.binarySearch(nums, 30);

// CORRECT
Arrays.sort(nums);  // sort first!
int idx = Arrays.binarySearch(nums, 30);  // now works
```

7. Creating an Array with Both Size and Values

```java
int[] arr = new int[3]{1, 2, 3};  // COMPILE ERROR

// Fix option 1:
int[] arr = new int[]{1, 2, 3};

// Fix option 2:
int[] arr = {1, 2, 3};

// Fix option 3:
int[] arr = new int[3];
arr[0] = 1; arr[1] = 2; arr[2] = 3;
```

QUICK REFERENCE — ARRAY DEBUGGING CHECKLIST

- **ArrayIndexOutOfBoundsException** → check loop bound (`<` not `⩽`), check empty arrays
- **Unexpected values after assignment** → you might have aliased instead of copied — use `Arrays.copyOf()`
- **Wrong binarySearch result** → array must be sorted first — verify with `Arrays.toString()`
- **for-each not modifying array** → for-each gives a copy for primitives; use indexed for loop to modify
- **NegativeArraySizeException** → size variable is negative — validate before creating array
- **Prints memory address** → use `Arrays.toString()` for 1D, `Arrays.deepToString()` for 2D+
- Print the array at every step with `Arrays.toString()` when debugging — it's the fastest way to see what's happening

Ref

Sources & References

This document was compiled from the following resources. All content was adapted and restructured for educational use.

[1] Programiz — Java Programming
Java Arrays tutorial: declaration, initialization, sorting, searching
https://www.programiz.com/java-programming/arrays

[2] GeeksforGeeks — Arrays in Java
Articles on Arrays.sort, Arrays.binarySearch, multidimensional arrays, System.arraycopy
https://www.geeksforgeeks.org/arrays-in-java/

[3] Oracle — Java Documentation — java.util.Arrays
Official API reference for all Arrays class methods
https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Arrays.html

[4] Baeldung — Java Arrays
Guides on array copying, sorting strategies, binary search usage
https://www.baeldung.com/java-arrays-guide

[5] W3Schools — Java Arrays
Quick reference on array syntax and multidimensional arrays
https://www.w3schools.com/java/java_arrays.asp

CREATED BY

Sepehr Ghardashi

sepehr.ghardashi@gmail.com